

1

a

i

Since all $\forall i, 0 \leq p_i \leq 1$. This means for some $k : 1 \leq k < n$ then $\prod_{i=1}^k p_i \leq \prod_{i=1}^{k+1} p_i$

Thus in a machine system, we run into a problem if we have all p_i be nonzero but we have multiple p_i near 0 or at the points where representing the smallest possible nonzero number on our machine. Then we end up with a truncation error and likely rounding down to 0.

It is important to note that since multiplication is a benign operation there is no issue in errors exploding out but instead the issues arise for truncation errors.

ii

Let $P = \sum_{i=1}^n \ln p_i$, then $e^P = \prod_{i=1}^n p_i$

This is better because since each p_i is bound between 0 and 1, then $\ln p_i$ must be less than or equal to 0. Thus even though we are using addition operators the addition is benign since all $\ln p_i$ are the same sign (negative). It is important to note that we have issues for $p_i = 0$ since $\ln 0$ is undefined and approaches $-\infty$

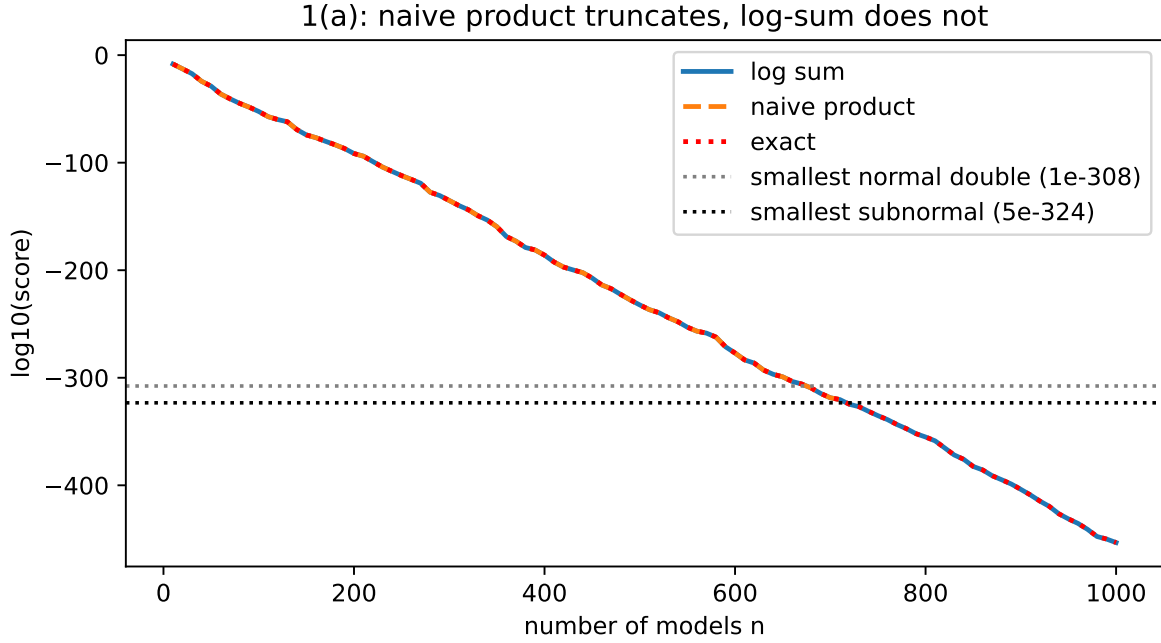
iii

So i did uniform random number generation for $p_i \in [0.00001, 1]$. The reason why I removed 0 from the interval is because then my solution would simply return negative infinity and the naive solution would be 0 and would essentially ruin the truncation issue I was talking about.

The random number generations were of sets size $\{100, 500, 750, 800, 1000\}$, corresponding to an expected values of $\approx 2^{-100}, 2^{-500}, 2^{-750}, 2^{-800}, 2^{-1000}$. Note that the latter 2 are actually beyond floating point representation in python thus should only be feasibly representable in log scale or some other means.

$ p = n$	naive multi- plication	$\log(\text{naive})$	$\log \text{ sum}$	exact (as log)	naive $\hat{e}$	$\log \text{ sum } \hat{e}$
100	$6.1223e^{-46}$	-45.2131	-45.2131	-45.2131	$3.191e^{-15}$	0
500	$7.2193e^{-219}$	-218.1415	-218.1415	-218.1415	$7.795e^{-15}$	$1.303e^{-16}$
750	$4.2310e^{-310}$	-309.3736	-309.3736	-309.3736	$1.878e^{-14}$	$1.837e^{-16}$
800	0	$-\infty$	-332.0385	-332.0385	1	$1.712e^{-16}$
1000	0	$-\infty$	-432.2933	-432.2933	1	$1.265e^{-16}$

iv



Above we see the exact points I was mentioning with running into the problem of eventually we get too small that the machine is not able to properly represent the numbers anymore. Thus the naive product will just truncate to 0 while the log sum is better able to represent these very very small numbers. To reiterate the reason why the naive method fails is for the case when $\prod_i p_i$ becomes too small to be represented by our machine. Instead we resort to an equivalent log sum which avoids this issue by summing negative numbers together. The log sum is able to represent many orders of magnitude more precision than our naive method simply by the limit of negative numbers is much larger than the smallest number we can represent.

b

i

Since x_i is not bound, then e^{x_i} can be beyond what we can represent on our machine is if $|x_i|$ is really large resulting in e^{x_i} being so large we cannot represent it or alternatively too small we can't represent it (depending on the sign of x_i). Note that if e^{x_i} is really small, then usually we do not care unless ALL x_i are small because then the denominator and numerator are really small. Instead the more dangerous case for this (and more likely) is a few x_i that are really large such that e^{x_i} is beyond what we are able to represent on the machine.

ii

We could instead change our exponents to prevent the top and bottom from exploding with the following:

$$\frac{e^{x_i}}{\sum_j e^{x_j}}$$

$$\frac{e^{-m} e^{x_i}}{e^{-m} \sum_j e^{x_j}}$$

Where $m = \max_i x_i$

$$\frac{e^{x_i - m}}{\sum_j e^{x_j - m}}$$

Since $m = \max_i x_i$ and we are finding the softmax, then

$$\frac{1}{\sum_j e^{x_j - m}}$$

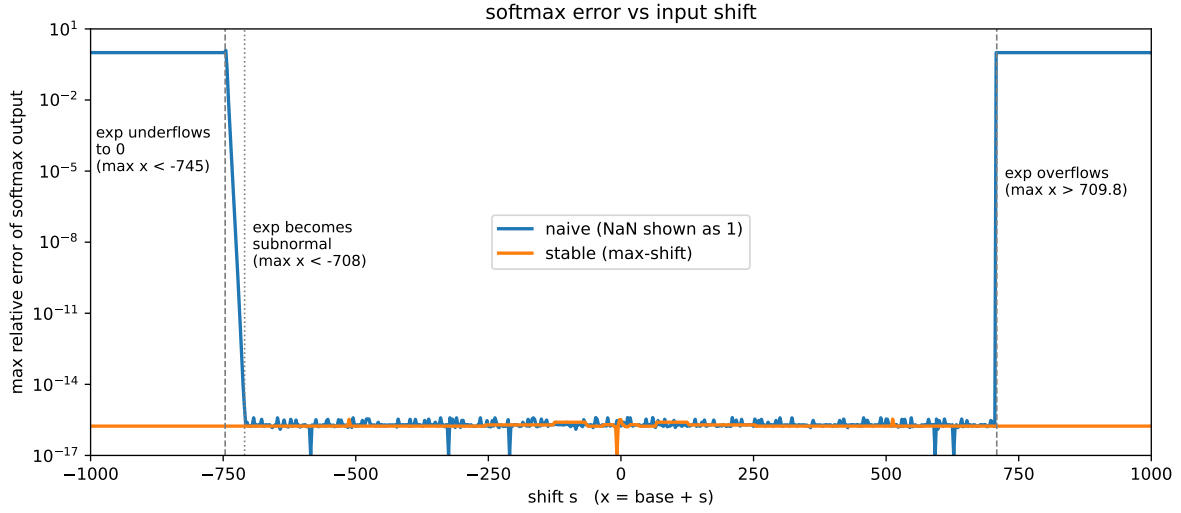
Thus with this shift, in the case of one really large x_i we shift everything down to avoid the few elements exploding. In the case of ALL really small elements then our shift actually bumps everything up and again we avoid the case of having too small numbers and getting a divide by 0 error.

iii

Since we only care about the maximum values. I did random normal sampling for 10 elements in $N(0, 1)$ and then applied various shifts s up and down with progressively increasing $|s|$ but changing the direction of the shift.

Shift s	Naive softmax e	Shifted softmax e
-1000	<i>nan</i>	$1.388e^{-17}$
- 750	<i>nan</i>	$1.388e^{-17}$
- 700	$6.939e^{-18}$	$1.388e^{-17}$
- 500	$5.551e^{-17}$	$6.939e^{-18}$
- 100	$5.551e^{-17}$	$5.551e^{-17}$
0	$6.939e^{-18}$	$5.551e^{-17}$
100	$5.551e^{-17}$	$5.551e^{-17}$
500	$5.551e^{-17}$	$6.939e^{-18}$
700	$5.551e^{-17}$	$1.388e^{-17}$
750	<i>nan</i>	$1.388e^{-17}$
1000	<i>nan</i>	$1.388e^{-17}$

iv



Here we see that error grows to 1 for naive softmax when x starts to get too big or too small. On the other hand, my proposed alternative method of shifting the elements resulted in consistently low errors. To reiterate, the reason why the naive method is prone to errors is because as $|x_i|$ becomes sufficiently large then we are unable to represent e^{x_i} with high precision and end up truncating e^{x_i} as 0 or ∞ depending on the sign of x_i . My proposed method avoids this by ensuring we find a numerically equivalent solution that does not rely on computing e^{x_i} for large x_i and avoids summing infinitely small numbers when all x_i are 0 by shifting the elements of x such that its max element is 0.

2

a

$$x = 2x - yx^2$$

$$\rightarrow yx^2 + x = 2x$$

$$\rightarrow yx^2 = x$$

$$\rightarrow yx = 1$$

$$\rightarrow x = \frac{1}{y}$$

$$\text{Or alternatively, } f\left(\frac{1}{y}\right) = \frac{2}{y} - \frac{y^2}{y} = \frac{2}{y} - \frac{1}{y} = \frac{1}{y}$$

b

Relative error $\hat{e}_n = \left| \frac{\frac{1}{y} - x_n}{\frac{1}{y}} \right| = |1 - yx_n|$

And e_{n+1} will be $|1 - yx_{n+1}|$ so

$$\rightarrow \hat{e}_{n+1} = |1 - y(2x_n - yx_n^2)|$$

$$\rightarrow \hat{e}_{n+1} = |1 - 2yx_n + y^2x_n^2|$$

$$\rightarrow \hat{e}_{n+1} = |(1 - yx_n)^2|$$

$$\rightarrow \hat{e}_{n+1} = (e_n)^2$$

Therefore we only converge when our relative error e_n is less than 1 because squaring decreases for decimal points, stays same at 1 and goes to infinity at greater than 1.

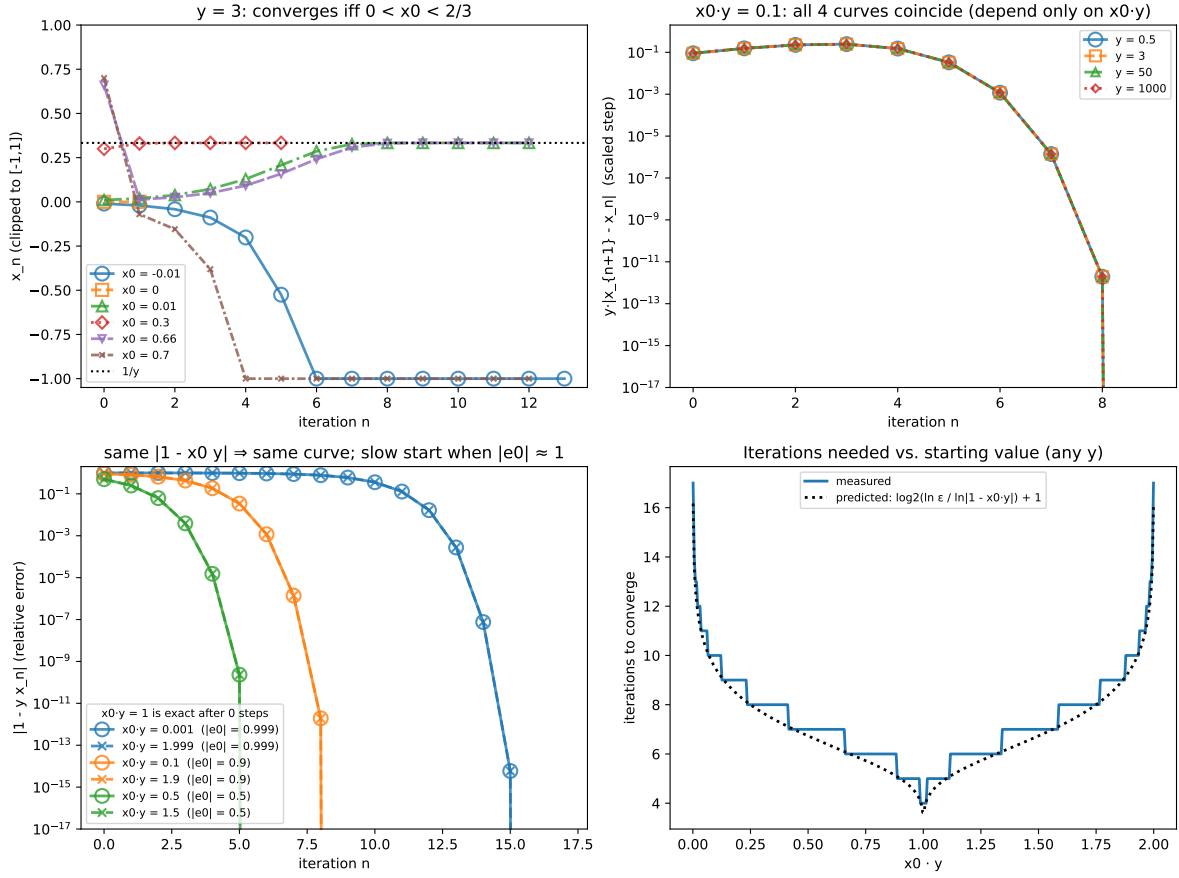
Note that this is for both positive and negative values

Therefore if $|1 - yx| < 1$ then we will converge. Thus the range of starting x is $0 < x_0 < \frac{2}{y}$

c

We have the exact case of quadratic convergence here because the relative error at each step is approximately the square of the relative error of the previous steps. So by the notes, the number of correct decimal digits doubles with each iteration.

d



First figure 1 demonstrates that we converge to $x_n = \frac{1}{y}$ when $x_0 \in (0, \frac{2}{y})$ because x_0 outside this range diverged (or converged to 0).

From the second figure, we notice that the change in relative error / the scaled steps between x_i and x_{i+1} depend solely on $x_0 * y$. This is because the figure shows 4 different starting y, x_0 pairs such that their products are identical and the resulting curves are also identical.

Figure 3 shows the symmetry of convergence demonstrating that the two starting values a, b that satisfy $|1 - x_0 y| = |\hat{e}_0|$ follow identical lines when $|\hat{e}_0| < 1$. This is because we show when error is ± 0.5 we follow the same path to convergence. Additionally figure 3 also demonstrates that convergence is slower when initial error is higher.

Additionally since figures 2 and 3 are relative errors, we can also see clearly that convergence is non linear and is in fact quadratic. To further emphasize this point figure 4 demonstrates that our theoretical quadratic convergence from **2.c** perfectly coincides with our observed convergence iterations.

3

a

$$x = x \frac{y}{2^x}$$

$$\rightarrow 1 = \frac{y}{2^x}$$

$$\rightarrow 2^x = y \text{ note i'll use this later in 3b}$$

$$\rightarrow x = \log_2 y$$

b

$$\text{Suppose } x^* = \log_2 y$$

$$e_n = x_n - \log_2 y = x_n - x^* \text{ for absolute error and } \hat{e}_n = \frac{x_n}{x^*} - 1 \text{ for relative error}$$

$$e_{n+1} = x_{n+1} - x^* \text{ for absolute error}$$

$$\rightarrow e_{n+1} = \frac{x_n y}{2^{x_n}} - x^*$$

$$\text{From 3a we know that } \frac{y}{2^{x^*}} = 1$$

$$\rightarrow e_{n+1} = \frac{x_n y}{2^{x_n}} - \frac{x^* y}{2^{x^*}}$$

$$\rightarrow e_{n+1} = y \left(\frac{x_n}{2^{x_n}} - \frac{x^*}{2^{x^*}} \right)$$

$$\rightarrow e_{n+1} = y \left(\frac{2^{x^*} x_n}{2^{x^*} 2^{x_n}} - \frac{2^{x^*} x^*}{2^{x^*} 2^{x_n}} \right)$$

$$\rightarrow e_{n+1} = y \left(\frac{2^{x^*} x_n - 2^{x^*} x^*}{2^{x^*} 2^{x_n}} \right)$$

$$\text{From 3a we know that } y = 2^{x^*}$$

$$\rightarrow e_{n+1} = \frac{2^{x^*} x_n - 2^{x^*} x^*}{2^{x_n}}$$

$$\rightarrow e_{n+1} = \frac{2^{x^*} x_n}{2^{x_n}} - x^*$$

$$\rightarrow e_{n+1} = 2^{x^* - x_n} x_n - x^*$$

$$\rightarrow e_{n+1} = 2^{-e_n} x_n - x^*$$

$$\text{Thus relative error is now: } \hat{e}_{n+1} = \frac{2^{-e_n} x_n - x^*}{x^*}$$

$$\rightarrow \hat{e}_{n+1} = \frac{2^{-e_n} x_n}{x^*} - 1$$

$$\rightarrow \hat{e}_{n+1} = \frac{x_n}{x^*} 2^{-\hat{e}_n x^*} - 1$$

$$x_n = x^* + e_n$$

$$\begin{aligned}
\rightarrow \hat{e}_{n+1} &= \frac{x^*+e_n}{x^*} 2^{-\hat{e}_n x^*} - 1 \\
\rightarrow \hat{e}_{n+1} &= (1 + \frac{e_n}{x^*}) x^* 2^{-\hat{e}_n x^*} - 1 \\
\rightarrow \hat{e}_{n+1} &= (1 + \hat{e}_n) 2^{-\hat{e}_n x^*} - 1 \\
\rightarrow \hat{e}_{n+1} &= (1 + \hat{e}_n) y^{-\hat{e}_n} - 1 \\
\rightarrow \hat{e}_{n+1} &= (1 + \hat{e}_n) e^{-\hat{e}_n \ln y} - 1 \\
\rightarrow \hat{e}_{n+1} &\approx (1 + \hat{e}_n)(1 - \hat{e}_n \ln y) - 1 \\
\rightarrow &= 1 - \hat{e}_n^2 \ln y + \hat{e}_n - \hat{e}_n \ln y - 1 \\
\rightarrow &= -\hat{e}_n^2 \ln y + \hat{e}_n(1 - \ln y) \\
\rightarrow &\approx \hat{e}_n(1 - \ln y)
\end{aligned}$$

Thus linear convergence for $|1 - \ln y| < 1$ because then $|e_n| > |e_{n+1}|$ when $|e_n| > 0$

If we solve this then actually convergence for $1 < y < e^2$

c

From 3.b, we have that we converge for values of y between 1 and e^2 so converges for values of x^* between 0 and $2 \log_2 e > 1$

This is from the lipschitz convergence for $|e_{n+1}| = \rho |e_n|$ converging for $|\rho| < 1$

Given $g(x) = x \frac{y}{2^x}$, then $g'(x) = \frac{y}{2^x}(1 - x \ln 2)$

Since $x \in (x^*, 1)$, $0 < \frac{y}{2^x} \leq \frac{y}{2^{x^*}} = 1$ and $0 < 1 - x \ln 2 < 1 - x^* \ln 2 < 1$

Thus $g'(x)$ is between 0 and 1 for the whole interval of $[x^*, 1]$ therefore we converge

d

$$0 < x^* < 1 \rightarrow 0 < \log_2 y < 1 \rightarrow 1 < y < 2$$

For decomposing y not in this range, since we are solving for $\log_2 y$ and this works for y between 1 and 2, we can generalize this into $k + \log_2 m$ where m is now bound between 1 and 2.

What this means for y is $\log_2 y = k + \log_2 m \rightarrow y = m 2^k$ which can represent all real numbers greater than 0. Essentially multiplying or dividing y until the result is an m between 1 and 2 and some integer k

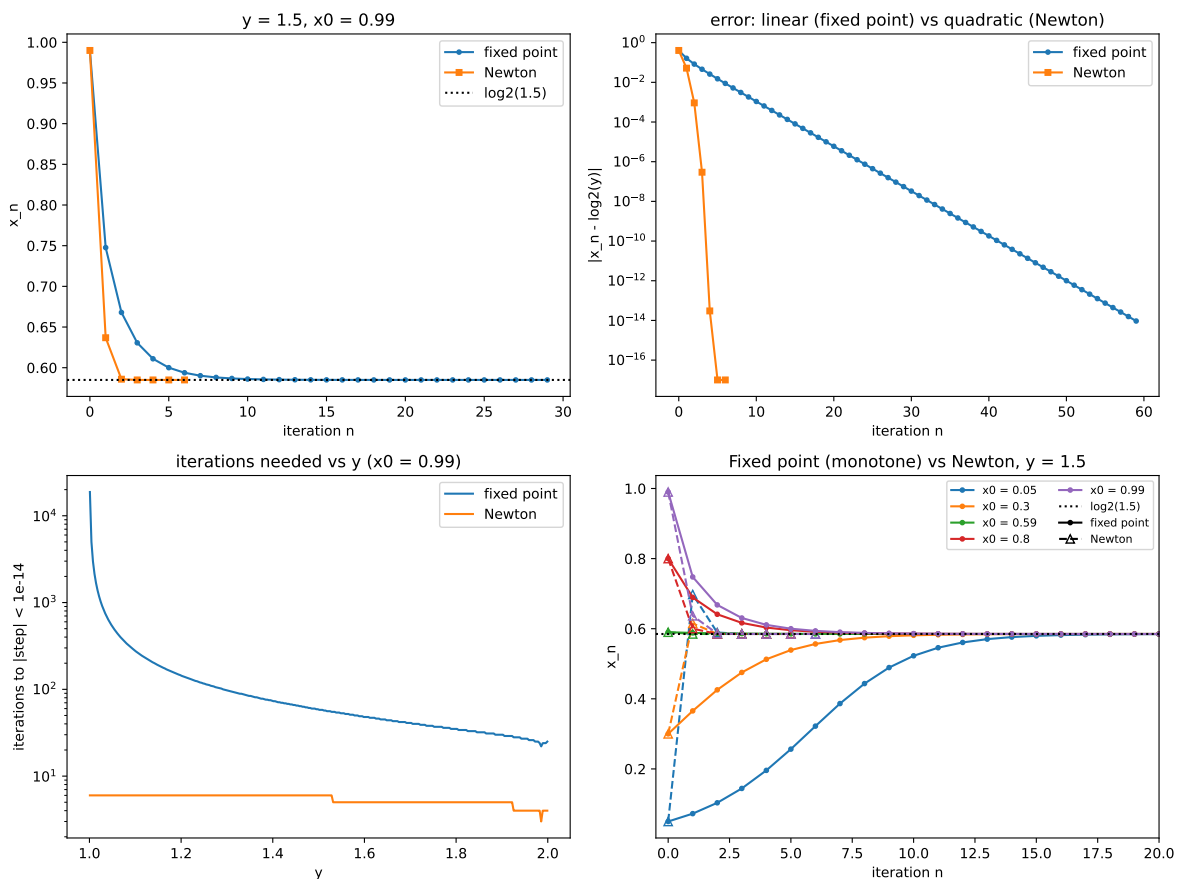
We know this algorithm converges because we know it converges for values of y between 1 and 2 and we can easily add /subtract back until we get our original desired number

e

$$f(x) = 2^x - y, f'(x) = 2^x \ln 2$$

$$\text{Thus } x_{n+1} = x_n - \frac{f(x)}{f'(x)} = x_n - \frac{2^x - y}{2^x \ln 2}$$

f



Fixed point vs. Newton for $y \in (1, 2)$, $x_0 = 0.99$, stop when $|x_{n+1} - x_n| < 10^{-14}$.

The fixed-point iteration count follows the predicted ρ : about $\ln(10^{-14})/\ln \rho$ iterations, which is 25 for $\rho \approx 0.31$ but 18,783 for $y = 1.001$ ($\rho = 0.999$). Its final error is also worse when $\rho \approx 1$, because a small step does not mean a small error: $|e_n| \approx |x_{n+1} - x_n|/(1 - \rho)$, which is $10^{-14}/0.001 = 10^{-11}$. Newton needs only 4–6 iterations for every y and reaches full double precision.

Error ratios for $y = 1.5$ (predicted: fixed-point ratio $\rightarrow 1 - x^* \ln 2 = 0.5945$; Newton $|e_{n+1}|/e_n^2 \rightarrow \ln 2/2 = 0.3466$):

Both ratios converge to the values derived in (c) and (e): the fixed-point method is linear, and Newton is quadratic.

Full range with range reduction (the final column is Python’s `math.log2` for reference):

Both methods give correct results across many orders of magnitude, including $y < 1$. Newton agrees with `math.log2` to all printed digits. The fixed-point method is off by up to $\sim 4 \times 10^{-13}$ in cases with small $\log_2 m$ (e.g. $m = 1.024$), again because of its slow rate there.

Figure 4. Top left: iterates for $y = 1.5$. Both approach $\log_2 1.5 \approx 0.585$ from above, monotonically, as proved in (c). Top right: error on a log scale. The fixed-point error is a straight line (linear convergence, slope $\log_{10} 0.5945$ per step); Newton’s curve bends downward (quadratic), reaching 10^{-16} in 5 steps vs. ~ 60 . Bottom left: iterations needed vs. y . The fixed-point count blows up as $y \rightarrow 1$ ($\rho \rightarrow 1$), while Newton stays at 4–6. Bottom right: iterates for $y = 1.5$ from several starting values; each color is one x_0 , with the fixed-point iteration as circles/solid and Newton as triangles/dashed. Fixed-point iterates starting above x^* decrease monotonically and those below increase monotonically. Newton reaches $\log_2 1.5$ in a handful of steps from every start, far sooner than the fixed-point iteration (from below the root, its first step overshoots above x^* , since $2^x - y$ is convex, and it then decreases to the root).

Summary. The given iteration is simple and guaranteed to converge on $1 < y < 2$ (after range reduction), but only linearly, with rate $1 - x^* \ln 2$, which is very slow for y close to 1. Newton’s method costs about the same per step (one 2^x evaluation) and converges quadratically, so it needs far fewer iterations (4–6 vs. 25–18,000+) and gives a more accurate result at the same tolerance.